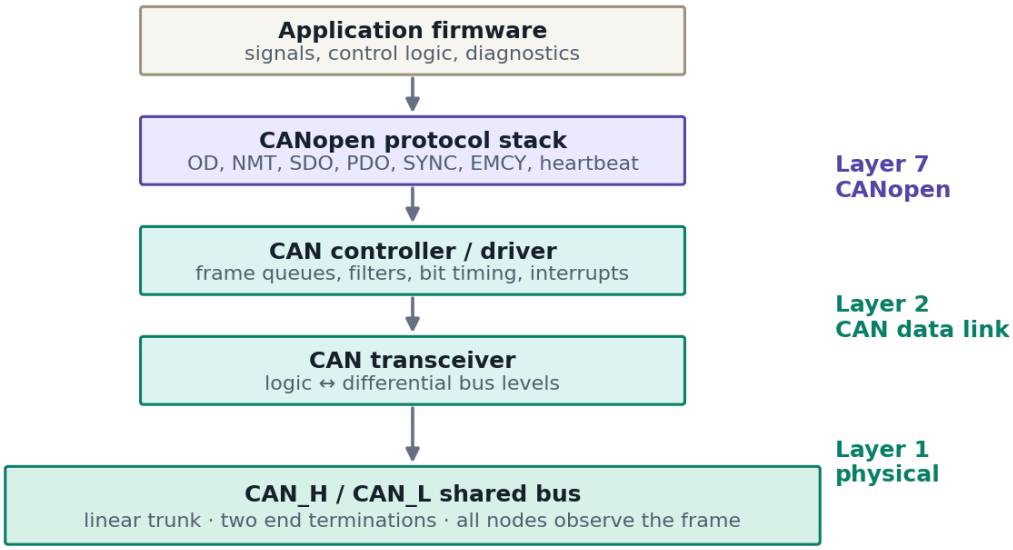
CAN & CANopen

Simple English Edition: From Basics to Advanced

Plain-English explanations, diagrams, practical firmware, troubleshooting and interview practice



Built for embedded firmware engineers
Learn one step at a time: what CAN is, how bits travel on the wires, how CANopen organizes messages, and how to build and debug embedded systems. Includes easy examples, diagrams, code patterns, message traces, exercises and references.

Source baseline: Siemens, *CANopen Tutorial*, Version 2.0, July 2019, Entry ID 109479771, supplemented with current CAN in Automation (CiA), Bosch and microcontroller-vendor documentation.

Plain-English revision: October 2026 | Examples are educational, not ready-to-use machine settings.

How to use this handbook

This book starts with the wires and CAN frames, then builds up to CANopen services, firmware design and advanced debugging. Read in order the first time. When a problem occurs, start with the physical layer and CAN controller, then move up to CANopen and the application. Code and values are examples; adapt them to your microcontroller and device manual.

Learning path: from beginner to advanced

Level	Read and practise
Beginner	Chapters 1-6: understand the bus, wiring, frames, arbitration and basic error handling.
Intermediate	Chapters 7-18: configure hardware, write safe C code, read/write Object Dictionary entries and use PDO, SDO, NMT and heartbeat.
Advanced	Chapters 19-27: device profiles, full network commissioning, timing/load analysis, security, safety and interview exercises.

The first terms to know

Term	Plain-English meaning
Node	One device connected to the CAN network.
CAN ID	The label on a CAN frame; it also affects which frame gets bus priority.
Node ID	A CANopen device number, normally 1-127 in CANopen CC. It is not the same as the CAN ID.
Object Dictionary (OD)	The device's organised list of parameters and application values.
PDO	A short, mapped message for live process data.
SDO	A request/response service used to read or write a device parameter.
NMT / heartbeat	NMT changes network state; heartbeat periodically reports that a node is present and its state.
EMCY	A CANopen message used to report a device error event.

Source and limits

The main source is the uploaded Siemens *CANopen Tutorial*, Version 2.0, July 2019 (38 pages), which focuses on CANopen CC over Classical CAN. This edition uses its topic sequence but updates explanations and checks details against current CiA, Bosch and ST documentation. The full specifications, product EDS/DCF, transceiver datasheet and MCU reference manual still take priority for real hardware.

Contents

1. The big picture: what, why and where CAN is used	2. System architecture and layers
3. Physical layer, wiring and termination	4. Classical CAN frames and identifiers
5. Arbitration and bus priority	6. CAN reliability and error confinement
7. Bit timing and controller configuration	8. Embedded C implementation patterns
9. CANopen architecture, profiles and variants	10. The Object Dictionary, EDS and DCF
11. COB-ID defaults and communication relationships	12. NMT, startup and state machine
13. PDO: real-time process data	14. SDO: parameter access and configuration
15. SYNC, TIME and deterministic exchange	16. EMCY, error register and history
17. Heartbeat and node monitoring	18. LSS node-ID and bit-rate configuration
19. Device profiles: CiA 401 and CiA 402	20. Complete example network and trace
21. Debugging and troubleshooting	22. Bus load, latency and system reliability
23. Security and functional safety	24. Interview question bank with answers
25. Practical exercises with worked solutions	26. Field checklists and revision sheets
27. Glossary and reference material	

1. The big picture: what, why and where CAN is used

In plain English: CAN lets many devices share a pair of wires. The frame ID labels the message and decides who gets priority.

Controller Area Network (CAN) lets electronic devices exchange short messages over a shared bus. It is common in vehicles, industrial machines, robots and motor controllers. Each frame has an identifier (ID). The ID labels the message and sets its priority; it does not always name one specific receiver. Every node can see the frame, then decide whether to use it. Any node may try to send when the bus is free, and arbitration decides which message goes first. [R2, R3]

Why it exists

- Reduce point-to-point wiring between controllers, sensors, actuators and supervisory units.
- Allow high-priority control/status messages to win bus access without destroying a competing frame.
- Detect common transmission faults using CRC, bit monitoring, bit stuffing, frame checks and acknowledgement.
- Support interoperable products when they share the same physical layer, bit rate, identifier plan and higher-layer protocol.

Typical use cases

Application	Typical CAN/CANopen data
Mobile machinery / industrial vehicles	Joystick commands, motor controller status, interlocks, battery/charger status, sensor readings
Factory automation	Remote I/O, drives, gateways, operator panels
Robotics and machines	Motion setpoints, actual position/velocity, diagnostics, state monitoring
Automotive / marine	ECU status and control, sensors, actuators
Embedded products	Configuration, firmware diagnostics, product-specific telemetry

CAN versus related interfaces

Interface	Main characteristic	Use / trade-off
UART	Point-to-point asynchronous byte stream	Simple debug links; no native multi-node arbitration
SPI	Clocked short-range master/peripheral bus	Fast local peripherals; chip-select and PCB-level wiring
I²C	Two-wire addressed board-level bus	Low-speed/medium-speed local sensors; pull-ups and capacitance constrain it
RS-485	Differential electrical layer, not one complete protocol	Longer industrial links; software/protocol defines framing and arbitration
Ethernet	High throughput, switching/routing ecosystem	Networked systems with larger data volumes; higher stack complexity
LIN	Low-cost scheduled automotive sub-bus	Simple sensors/actuators with a schedule and one master
CAN	Prioritized multi-master message bus with strong error handling	Control/status messages in electrically noisy systems
CANopen	Higher-layer rules on top of CAN CC	Standardized objects, services, node states and device profiles

Remember: CAN is the communication foundation (data-link and physical layers plus related protocol rules). CANopen defines how application parameters, process data, network state, configuration and diagnostic services are organized above CAN. Having a working CAN wire does not by itself make two devices CANopen-compatible.

2. System architecture and layers

In plain English: Application code creates the data, CANopen organizes it, the CAN controller handles frames, and the transceiver connects the controller to the wires.

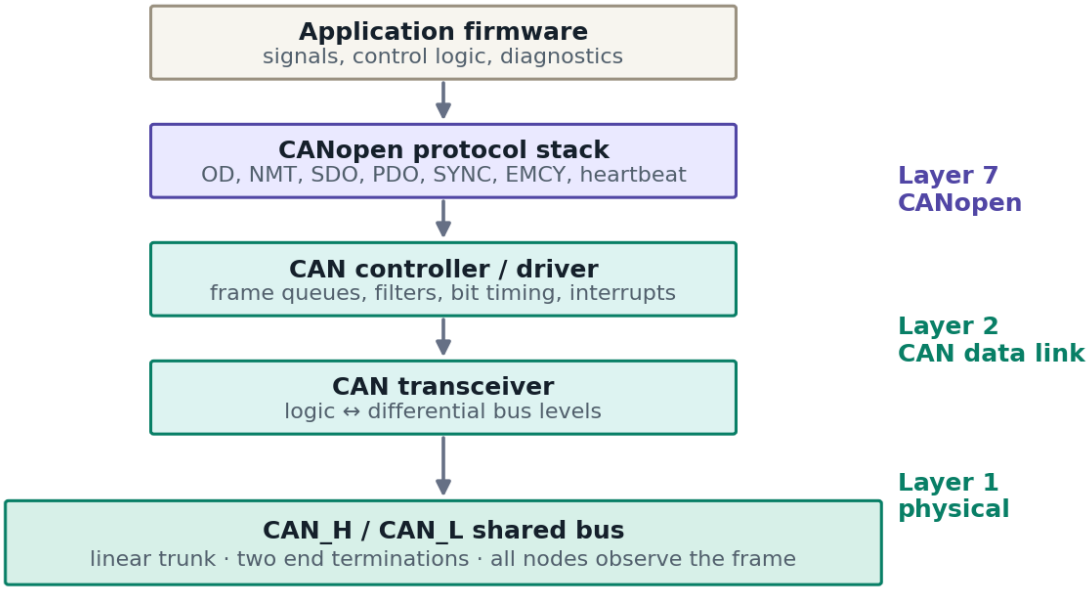


Figure 1. The full path from application intent to the shared physical bus. The exact separation between firmware and controller hardware depends on the microcontroller and CAN stack.

What each layer does

Component	Responsibility	What can fail
Application	Decides what to measure, command, interlock and report	Wrong scaling, stale value, unsafe state machine
CANopen stack	Implements OD access, PDO mapping, SDO transfers, NMT and supervision services	Wrong COB-ID, unsupported object, incorrect NMT behavior
CAN driver / hardware abstraction layer (HAL)	Initializes peripheral, queues frames, dispatches receive/error events	Race conditions, lost interrupts, queue overflow
CAN controller	Forms/checks CAN frames, arbitration, CRC, ACK/error handling, filtering, timing	Bit-rate mismatch, bus-off, bad filter configuration
CAN transceiver	Converts controller logic levels into differential bus signals and back	Standby pin, wrong voltage family, damaged transceiver
Bus cabling and termination	Provides valid signal path and impedance environment	Open/short, reflection, poor ground reference, wrong termination

A useful debugging rule is to move from bottom to top: physical waveform and controller errors first, CAN identifiers and frame reception second, CANopen service/state third, application interpretation last. Debugging the application before confirming the lower layers can waste time.

3. Physical layer, wiring and termination

In plain English: A typical high-speed CAN network is one main cable with short branches and one 120-ohm resistor at each end.

For common high-speed CAN, two wires carry the signal: CAN_H and CAN_L. The receiver looks at the voltage difference between the wires, not just one wire against ground. This helps reduce the effect of some electrical noise. The exact voltage limits come from the chosen transceiver datasheet and physical-layer standard. [R4, R5]

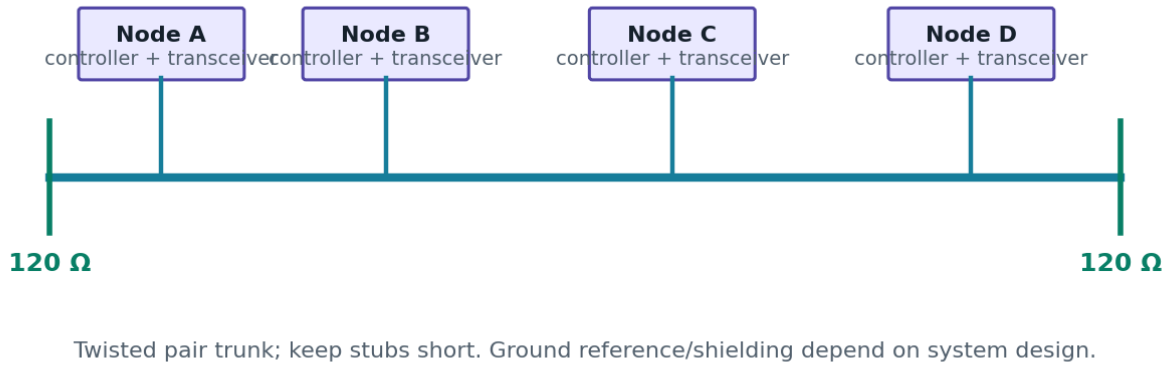


Figure 2. Typical linear trunk with short stubs and one 120-ohm terminator at each physical end. Do not add a terminator at every node.

Termination and resistance check

With the network powered down and isolated as appropriate, a common quick check on a conventional high-speed CAN segment is about 60 ohms measured between CAN_H and CAN_L, because two 120-ohm terminators are in parallel. This is only a sanity check: connected electronics, split termination, other network segments or nonstandard physical layers can change what an ohmmeter reads. Follow machine safety procedures and isolate power before resistance measurements.

Practical wiring checklist

- Use the transceiver-recommended cable and connector pinout; check CAN_H/CAN_L polarity.
- Terminate at the two ends of the physical trunk, not at arbitrary mid-span nodes.
- Keep stubs short relative to the bit time and cable propagation delay. Long star branches produce reflections; use a suitable active/passive topology only when designed for that physical layer.
- Check ground/reference potential, shields and chassis bonding according to the system design; do not assume shield is the signal return.
- Inspect supply and transceiver standby/enable pins. A perfect firmware configuration cannot communicate through a transceiver held in standby.
- If traffic fails only at higher bit rates or with one particular machine layout, inspect waveform ringing, edge quality, cable lengths, stub lengths and termination before rewriting the protocol stack.

Bit rate versus length

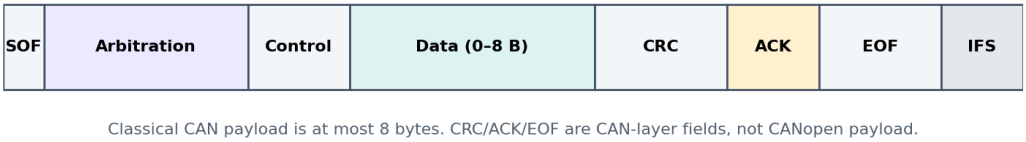
The common high-speed CAN family supports up to 1 Mbit/s under suitable segment-length and timing conditions. Increasing bit rate reduces the time available for signal propagation and settling, so the allowed trunk and stub lengths usually shrink. There is no single universal length table: cable characteristics, transceiver delays, oscillator tolerance, topology and selected bit timing all matter.

4. Classical CAN frames and identifiers

In plain English: A CAN frame is the packet on the wire. The ID and data bytes are different things, and the protocol or mapping explains the bytes.

This chapter describes the Classical CAN data frame. It should not be confused with the CANopen payload inside that frame. Classical CAN supports standard 11-bit identifiers and extended 29-bit identifiers. A Classical CAN data frame carries 0 to 8 data bytes. [R3, R6]

11-bit ID + RTR (standard frame); extended frames add ID bits and arbitration/control details



Classical CAN payload is at most 8 bytes. CRC/ACK/EOF are CAN-layer fields, not CANopen payload.

Figure 3. Simplified standard-format Classical CAN frame. Extended-format fields and bit-stuffing details are omitted from the picture.

Field	Purpose
Start of Frame (SOF)	One dominant bit marks frame start and helps synchronization.
Arbitration	Identifier and format/control bits determine priority; Remote Transmission Request (RTR) distinguishes data from legacy remote frames in Classical CAN.
Control	Includes identifier-extension and reserved/control information and the Data Length Code (DLC). Exact bit layout depends on standard/extended format.
Data	0–8 bytes in Classical CAN. CANopen places PDO/SDO/NMT/etc. information here according to the service.
CRC	Cyclic redundancy check sequence plus delimiter; receiver checks frame integrity.
ACK	Transmitter sends recessive in ACK slot; any node that correctly receives the frame can drive dominant to acknowledge it at CAN data-link level.
EOF / intermission	End-of-frame sequence and bus-free spacing before a subsequent frame can start.

Example: decode a generic Classical CAN frame

```
ID=0x321, DLC=4, DATA=2A 00 10 01
```

The ID 0x321 is an 11-bit identifier because it is less than 0x800. DLC 4 means four payload bytes. The bytes have no universal meaning until the higher-layer protocol or application mapping defines them. For CANopen PDO, mapping determines the meaning; for SDO, bytes include command and OD address fields; for NMT, the payload has a standardized command and node ID.

Important distinction: CAN ACK is not an application-level acknowledgement. An ACK only says at least one active receiver found the frame valid at CAN level; it does not prove the intended CANopen server processed the request or that the application completed the command. SDO has its own request/response protocol.

Other Classical CAN frame types

Frame type	Plain-English purpose
Data frame	Carries data bytes. This is the usual frame used by CANopen.
Remote frame	A legacy Classical CAN request asking another node to send data. CAN FD does not support remote frames.
Error frame	The controller signals that it detected a CAN protocol error; the current frame is aborted and may be retried.
Overload frame	A legacy CAN mechanism for delaying the next frame. Application firmware normally does not create it directly.

5. Arbitration and bus priority

In plain English: When two nodes start together, dominant zero beats recessive one. Under the same frame-format conditions, the lower ID wins.

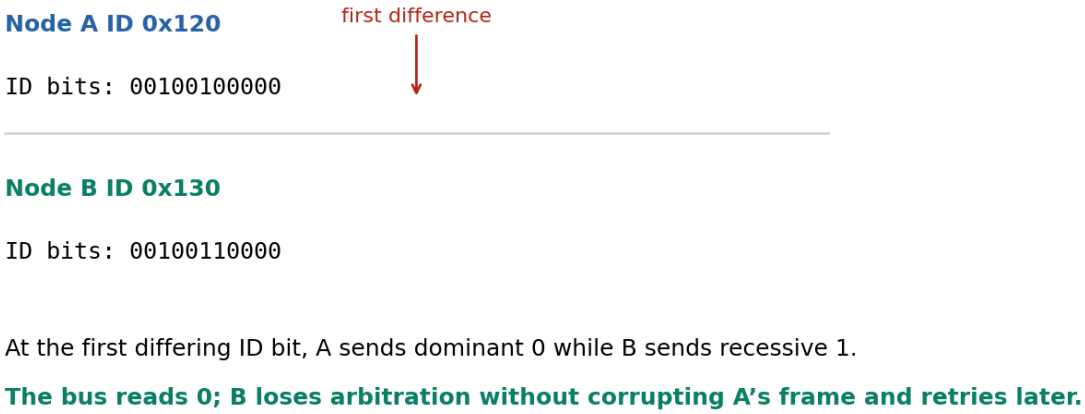


Figure 4. Two competing standard identifiers. At the first differing identifier bit, the transmitter sending recessive loses when it observes dominant. The frame already on the bus remains valid.

How arbitration works

- Nodes wait for a bus-idle opportunity, then may begin transmitting.
- During arbitration each node monitors the bus while sending. Dominant 0 overrides recessive 1.
- If a node sends recessive but observes dominant in the arbitration field, it withdraws without emitting an error; it becomes a receiver and can retry later.
- Under matching frame-format conditions, the lower numerical identifier has higher priority. Other format-bit interactions matter when standard and extended formats compete.
- The identifier represents message identity and priority, not inherently the sender node. A CANopen predefined COB-ID often embeds a node ID, but that is a higher-layer convention.

Priority has a cost

High-priority traffic can repeatedly win and delay lower-priority traffic. A design should bound periodic traffic, calculate worst-case message latency, avoid excessive high-priority bursts, and document identifiers centrally. Assign the highest priorities to messages with genuine deadline/safety significance, not simply to messages that are easiest to debug.

6. CAN reliability and error confinement

In plain English: CAN checks many transmission errors and can stop a faulty controller from transmitting. These checks do not prove that data is safe or authentic.

CAN combines several checks so nodes can detect many transmission errors and confine faulty participants. These mechanisms improve data-link integrity, but they do not authenticate the sender and do not guarantee that application data is fresh or safe. [R3, R6]

Mechanism	What it detects / does	Typical symptom or clue
CRC check	Checks frame bit sequence for transmission corruption	Error frames/retransmissions; often wiring/noise/timing
Bit monitoring	Transmitter compares observed bus level to its own transmitted bit, with defined exceptions	Bit error, especially with wiring or competing-transmitter problems
Bit stuffing	Detects invalid stuffing patterns in the stuffed portion of the frame	Stuff error, waveform or timing issue
Frame/form check	Checks required fixed/delimiter bits and frame structure	Form error / invalid frame fields
ACK check	Detects absence of a dominant ACK in the ACK slot	Transmit ACK error; no active receiver, wrong bit rate, wiring or filters are not necessarily the cause
Error counters / confinement	Controller tracks transmit and receive errors and enters error-active, error-passive, then potentially bus-off behavior	Rising TEC/REC, error-passive, bus-off

Error-active, error-passive and bus-off

Error-active nodes participate normally and may signal errors with active error flags. Error-passive nodes have sufficiently high error counts and signal errors passively; their participation is restricted by protocol rules. Bus-off prevents a node from transmitting on the bus after its transmit error count reaches the protocol threshold. Exact status registers and recovery controls are controller-specific.

Recovery policy should be explicit

Do not blindly reset a bus-off controller in a tight loop. Record the reason, timestamp, TEC/REC, last successfully transmitted/received ID and application state. Check physical wiring and timing, then apply the controller/documented recovery procedure. Add a bounded retry or maintenance state, and prevent a recovered communication link from automatically re-enabling hazardous motion without application-level revalidation.

7. Bit timing and controller configuration

In plain English: Bit timing tells each node when to read a bit. A matching bit-rate number alone does not guarantee enough timing margin.

The CAN controller divides each bit into small time steps called time quanta (TQ). It uses them to place the sample point, where the receiver reads the bit, and to adjust timing when edges arrive. A clock divider creates the TQ clock. The bit rate depends on the CAN peripheral clock, divider and number of TQ in each bit. Register names and values differ by controller.

Worked example (illustrative only)

Worked example (illustrative): assume an 80 MHz CAN peripheral clock, a clock divider of 10, and 16 TQ per bit. The TQ clock is 80 MHz / 10 = 8 MHz, so one TQ is 125 ns. A bit lasts 16 × 125 ns = 2 µs, so the bit rate is 500 kbit/s. If the bit is sampled after 13 TQ, the sample point is 13/16 = 81.25%. The divider of 10 is the mathematical divider, not necessarily the number to write into a register. Check the controller manual for register encoding and allowed segment values.

Nominal bit timing configuration steps

- Identify the true CAN peripheral clock; do not assume the CPU clock equals the CAN clock.
- Choose the network nominal bit rate, then calculate total TQ and sample point within the MCU controller’s allowed ranges.
- Include transceiver propagation delay, cable delay, node oscillator tolerance and resynchronization limits.
- Configure all nodes compatibly; matching the displayed bit-rate number alone may not be sufficient if timing margins are poor.
- For CAN FD, configure nominal/arbitration timing separately from data-phase timing; BRS requires compatible nodes and physical-layer design.
- Validate with an actual trace and oscilloscope where possible. Use vendor calculators only for the exact controller family and verify against its reference manual.

Acceptance filters

Hardware acceptance filters reduce the frames delivered to firmware, but their syntax and mask/list behavior are controller-dependent. During commissioning, a permissive filter can simplify initial visibility; production configuration should be intentional. A too-restrictive filter can make a correctly transmitted frame appear missing in firmware, even when a bus analyzer sees it.

8. Embedded C implementation patterns

In plain English: Keep interrupt work short, use bounded queues, check every payload length, and process messages in a controlled task.

Separate low-level CAN frame handling from CANopen services and application logic. Keep interrupt routines short: capture the frame/status, move it to a bounded queue or ring buffer, and notify a task. Decode SDO/PDO/NMT in a protocol task or well-bounded callback. Avoid unbounded loops and blocking transmissions in interrupt context.

Generic frame structure

```
#include <stdint.h>
#include <stdbool.h>

#define CAN_CLASSIC_MAX_DLEN 8u
#define CAN_RX_ISR_BUDGET 4u   /* maximum frames handled in one interrupt */

typedef struct {
    uint32_t id;          /* raw CAN identifier */
    uint8_t  dlc;         /* validated payload length: 0..8 for Classical CAN */
    bool     extended;    /* false = 11-bit ID, true = 29-bit ID */
    bool     rtr;          /* Classical CAN remote-frame flag if supported */
    uint8_t  data[CAN_CLASSIC_MAX_DLEN];
    uint32_t timestamp_ms;
} can_frame_t;

/* Generic pattern; replace with vendor HAL calls. */
bool can_send(const can_frame_t *f, uint32_t timeout_ms);
bool can_receive(can_frame_t *f, uint32_t timeout_ms);
```

Safe receive path

```
void CAN_RxISR(void)
{
    can_frame_t f;
    for (uint8_t n = 0; n < CAN_RX_ISR_BUDGET; ++n) {
        if (!hal_can_read_fifo(&f)) {         /* exact API is vendor-specific */
            break;
        }
        if (f.dlc > CAN_CLASSIC_MAX_DLEN) {  /* defensive validation */
            diag_bad_dlc++;
            continue;
        }
        if (!rx_queue_push_from_isr(&f)) {
            diag_rx_queue_overflow++;
        }
    }
    notify_can_task_from_isr();
}

void CAN_Task(void)
{
    can_frame_t f;
    for (;;) {
        if (rx_queue_pop(&f, 10u)) { /* bounded wait so error status is checked periodically */
            canopen_dispatch_frame(&f);      /* stack validates ID/service */
        }
        if (hal_can_bus_off()) {
            record_bus_off(hal_can_tec(), hal_can_rec());
            request_bounded_recovery();
        }
    }
}
```

Periodic sensor PDO pattern

```
void publish_sensor(int16_t raw_temperature)
{
    /* Example mapping only: signed 16-bit value, little-endian. */
    can_frame_t f = {0};
    f.id = 0x181;       /* example TPDO1 for node ID 1 */
    f.dlc = 2;
    f.data[0] = (uint8_t)(raw_temperature & 0xFF);
    f.data[1] = (uint8_t)(((uint16_t)raw_temperature >> 8) & 0xFF);
    if (!can_send(&f, 5u)) {
        diag_tx_timeout++;
    }
}
```

Production review points

- Check DLC before accessing data bytes. In C, do not read data[1] unless dlc ≥ 2.
- Define endianness, signedness, scale, units, valid range and freshness for every application signal.
- Use bounded queues, explicit overflow counters and a policy for stale values.
- Use critical sections or lock-free primitives appropriate to the RTOS/MCU when sharing state between ISRs and tasks.
- Do not assume send API success means the application consumer acted. Check controller completion, CAN ACK where available, and CANopen/service/application acknowledgements separately.
- Keep vendor-specific registers and HAL calls out of portable protocol logic. Match controller clock and API semantics to official reference documentation.

9. CANopen architecture, profiles and variants

In plain English: CANopen adds common message types, device parameters, startup rules and diagnostics on top of CAN.

CANopen is a standardized higher-layer protocol for embedded control over CAN. In CANopen CC, it defines common objects, services and behavior above Classical CAN: configuration/diagnostics through SDO, real-time data through PDO, network state via NMT, plus synchronization, emergency reporting and error-control services. [R1, R7]

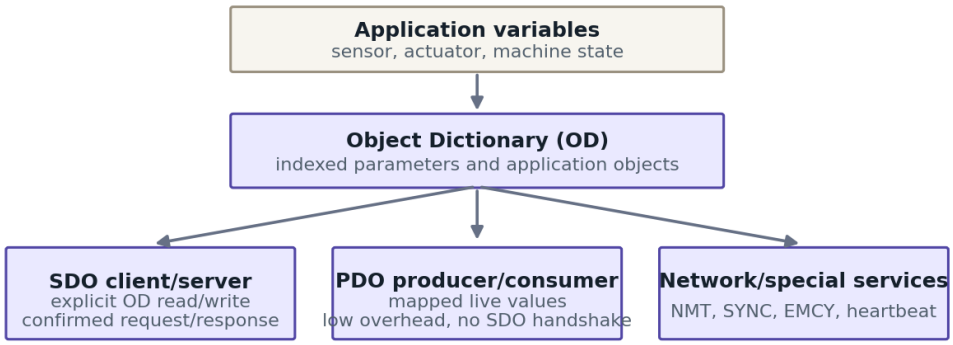


Figure 5. Different services use different data paths; not every frame contains an OD address.

Key concepts

- **Node ID:** logical identifier assigned to a CANopen device, normally 1..127 for CANopen CC. It is not identical to the raw CAN identifier.

- **COB-ID:** communication object identifier carried in the CAN identifier field. In common default mappings it combines function class and node ID; some COB-IDs are fixed and many are configurable.
- **Object Dictionary (OD):** indexed collection of communication parameters and application data.
- **Communication profile:** common protocol behavior and OD definitions, classically CiA 301 for CANopen CC.
- **Device profile:** standardizes an application class such as generic I/O or drives; it improves interoperability only within declared features and supported options.
- **CANopen manager / NMT manager:** coordinates network startup and NMT state; devices commonly implement the NMT server/state machine.

CANopen CC versus CANopen FD

Feature	CANopen CC	CANopen FD
Lower layer	Classical CAN / CAN CC	CAN FD
PDO application data	Up to 8 bytes per PDO frame	Up to 64 bytes per PDO frame; legal CAN FD lengths and padding matter
SDO family	SDO upload/download, segmented/block variants depending on support	USDO is a defining CANopen FD service; do not assume CC SDO semantics map one-to-one
Remote frames / guarding	Legacy node/life guarding exists in older CC systems; heartbeat preferred	CAN FD does not support remote frames; CANopen FD uses heartbeat/boot-up error control
Compatibility	CC controller and CC transceiver/network	Requires CAN FD-capable controllers and compatible network design; CANopen FD is a separate protocol variant

Do not treat CANopen CC and CANopen FD as interchangeable. A Classical-CAN-only node can misinterpret CAN FD traffic if inserted into a network that is not designed for safe coexistence. Validate the whole system’s controller, transceiver, topology, filters, bit rates and protocol stack. [R8, R9]

Where CAN XL fits

CAN XL is a newer CAN generation, separate from Classical CAN and CAN FD. Bosch describes CAN XL frames with payloads up to 2,048 bytes and data-phase bit rates up to 20 Mbit/s. Do not assume that a CANopen CC or CANopen FD stack can simply run over CAN XL; the higher-layer protocol and hardware support must explicitly match. [R18]

10. The Object Dictionary, EDS and DCF

In plain English: The Object Dictionary is like a device parameter book. EDS describes the device; DCF records a configuration.

Think of the Object Dictionary as the device's parameter book. It stores communication settings and application values. An entry is addressed by a 16-bit index and an 8-bit sub-index. The index selects an object; the sub-index selects a value inside a record or array. The CAN ID that carries an SDO request is different from this Object Dictionary address. [R1, R7]

Index range (common CANopen CC organization)	Purpose
0x0000	Reserved
0x0001–0x009F	Static and complex data types
0x1000–0x1FFF	Communication-profile parameters and standard communication objects
0x2000–0x5FFF	Manufacturer-specific application parameters
0x6000–0x9FFF	Standardized device-profile application parameters
0xA000–0xFFFF	Reserved / profile-dependent according to the applicable specification

Useful common objects

Index	Common meaning	Engineering use / caveat
0x1000	Device Type	Indicates device class/profile information; verify the device manual
0x1001	Error Register	Bit-coded summary of error categories
0x1003	Pre-defined Error Field	Error history; exact supported depth/behavior must match the implementation/specification
0x1005	COB-ID SYNC	SYNC identifier/producer-related configuration
0x1006	Communication Cycle Period	Configured SYNC cycle period where supported
0x1007	Synchronous Window Length	Synchronous PDO window where supported
0x1014	COB-ID EMCY	Emergency message identifier configuration
0x1017	Producer Heartbeat Time	Heartbeat producer period in milliseconds in standard CANopen CC implementations
0x1018	Identity Object	Vendor ID, product code, revision, serial number
0x1200+	SDO server parameters	COB-IDs and channel parameters; ranges depend on channel number
0x1400–0x15FF	RPDO communication parameters	Incoming PDO COB-ID, transmission type and related parameters
0x1600–0x17FF	RPDO mapping parameters	Define the meaning/bit length of incoming PDO bytes
0x1800–0x19FF	TPDO communication parameters	Outgoing PDO COB-ID, triggering/transmission type and related parameters
0x1A00–0x1BFF	TPDO mapping parameters	Define which OD values are packed into outgoing PDOs

Access and data types

The device description determines whether an object is read-only, writable, constant, mapped to PDO, or subject to state-dependent restrictions. CANopen uses standardized data types such as UNSIGNED8/16/32, INTEGER8/16/32, BOOLEAN, REAL types and visible strings, but availability and supported access vary by device. Never infer type from a value's appearance in a trace.

EDS and DCF

Electronic Data Sheet (EDS) describes the device's supported objects, data types, access, default parameters and communication capabilities. **Device Configuration File (DCF)** includes configuration values for a specific configured device/network, typically generated by configuration software. An EDS is not proof that every option is available in the installed firmware revision; compare vendor, product, revision and device manual. [R1]

11. COB-ID defaults and communication relationships

In plain English: A COB-ID labels a CANopen communication object. Default IDs are a starting point; the actual device configuration is the final answer.

For the default 11-bit CANopen CC connection set, many COB-IDs follow a predefined function-code plus node-ID convention. The table below contains the commonly used default identifiers. Treat each as a default and confirm the configured value in the device OD/EDS/DCF and actual trace. Some services such as NMT have fixed identifiers; PDO and certain other identifiers can be configured. [R1, R10]

Object	Default COB-ID	Typical purpose
NMT	0x000	Network-management command, two data bytes: command and target node ID; node ID 0 means broadcast
SYNC	0x080	Broadcast synchronization; default frame has no data
EMCY producer	0x080 + node ID	Emergency event reported by a node
TPDO1	0x180 + node ID	Device transmits mapped process data
RPDO1	0x200 + node ID	Device receives mapped process data
TPDO2 / RPDO2	0x280 / 0x300 + node ID	Second default transmit/receive PDO groups
TPDO3 / RPDO3	0x380 / 0x400 + node ID	Third default transmit/receive PDO groups
TPDO4 / RPDO4	0x480 / 0x500 + node ID	Fourth default transmit/receive PDO groups
SDO server-to-client response	0x580 + server node ID	Response from the node whose OD is accessed
SDO client-to-server request	0x600 + server node ID	Request addressed to the SDO server
Heartbeat / boot-up	0x700 + producer node ID	Error-control state/boot-up message

For node ID 5, default examples include TPDO1 = 0x185, RPDO1 = 0x205, SDO request to node 5 = 0x605, SDO response from node 5 = 0x585, and heartbeat = 0x705. These are identifier arithmetic examples, not evidence that a particular device supports or enables those objects.

Communication relationships

Model	Typical CANopen use	Acknowledgement concept
Client/server	SDO access to a server OD	Protocol response confirms request/result at service level
Producer/consumer	PDO, SYNC, EMCY and heartbeat distribution	One producer transmits; consumers may listen; PDO has no application-level per-consumer acknowledgement
NMT control	Manager sends state command to one or all nodes	State change is observed through later behavior/monitoring, not an SDO-style response for each NMT command

12. NMT, startup and state machine

In plain English: NMT controls a node's network state. PDO communication is disabled in pre-operational and normally enabled in operational.

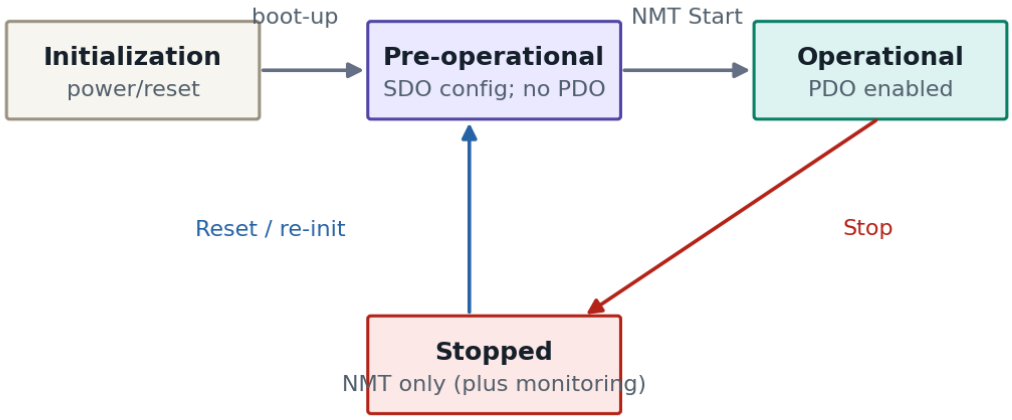


Figure 6. Simplified CANopen CC NMT state machine. A boot-up message is sent as initialization completes and the node enters pre-operational.

State	Typical behavior
Initialization	Internal device initialization; boot-up is sent as the final action before pre-operational.
Pre-operational	SDO configuration/diagnostics available; PDO communication disabled. Other supported services can be configured/active according to rules.
Operational	PDO process-data communication enabled; SDO and applicable other communication services remain available.
Stopped	Normal application communication is restricted; NMT and configured error-control monitoring behavior may remain.

NMT command bytes

Command specifier	Meaning
0x01	Start remote node / transition to operational
0x02	Stop remote node
0x80	Enter pre-operational
0x81	Reset node
0x82	Reset communication
Target node ID = 0	Broadcast to all nodes; otherwise targets one node

Typical startup



A typical commissioning sequence; some devices have fixed mappings or application-specific start conditions.

Figure 7. Typical, not universal, startup sequence. Application-specific checks can add steps before NMT Start.

Commissioning logic

- Power up and initialize MCU clocks, GPIO, transceiver enable and CAN peripheral.
- Verify bus health and observe expected boot-up frames on 0x700 + node ID, payload 0x00.
- Read identity object 0x1018 and verify device revision / expected node ID.
- Use SDO to inspect critical parameters and configure PDO communication/mapping only when supported and permitted.
- Start node via NMT; confirm the expected operational behavior via heartbeat, PDO traffic and application status.
- If a required node is absent, block dependent motion/control and report a diagnosable fault instead of assuming the network is healthy.

13. PDO: real-time process data

In plain English: PDO sends already-mapped live values with little extra protocol data. Both sides must agree on byte order, size, scale and meaning.

A Process Data Object (PDO) is a short CANopen message for live values, such as sensor readings, drive commands or status. The sender and receiver agree beforehand what each byte means. The PDO payload does not contain an Object Dictionary index/sub-index; the mapping tells the receiver how to decode the bytes. A CANopen CC PDO carries up to 8 data bytes. [R11]

Important: “PDO is unacknowledged” means there is no separate CANopen reply from each consumer. The CAN frame still uses the normal CAN-layer ACK slot. Keep these two kinds of acknowledgement separate when debugging.

Two configuration records per PDO

OD range	Record	What it defines
0x1400–0x15FF	RPDO communication parameters	Incoming COB-ID, transmission type, timer/inhibit settings where supported
0x1600–0x17FF	RPDO mapping parameters	Which OD values and bit lengths are expected in incoming PDO payload
0x1800–0x19FF	TPDO communication parameters	Outgoing COB-ID, triggering/transmission type, timers where supported
0x1A00–0x1BFF	TPDO mapping parameters	Which OD values and bit lengths are packed into outgoing PDO payload

These are the standard index ranges defined for PDO communication and mapping records. A device may implement only some of the records, so the range alone does not prove that a particular PDO exists or can be changed. [R11]

PDO transmission styles

Style	Behavior	Design note
Synchronous cyclic	A PDO is sent in relation to SYNC, often every Nth SYNC according to transmission type	Supports coordinated sampling/control; latency includes waiting for SYNC and arbitration
Synchronous acyclic	An event occurs, with transmission coordinated relative to SYNC	Useful for events that should be aligned to network cycle
Asynchronous event-driven	A change/event triggers a transmission	Bound event bursts and avoid excess bus load
Timer/event timer	A configured timer triggers updates where supported	Use a heartbeat/watchdog separately if detecting a missing producer is required
RTR / remote request	Legacy Classical CAN remote-frame mechanisms exist in some configurations	Not supported by CAN FD; CiA does not recommend remote-frame-based node/life guarding for new designs

PDO example: decode the bytes

Worked PDO payload example

Assume a sensor node has node ID 5 and sends TPDO1 using default COB-ID 0x185. Illustrative mapping: signed 16-bit temperature in 0.1 °C units, followed by unsigned 16-bit speed in rpm. All values are little-endian for this example only. If temperature = 23.4 °C, raw = 234 = 0x00EA; speed = 1500 = 0x05DC. The payload is `EA 00 DC 05`, DLC = 4.

Field	Bytes	Decode
Temperature	EA 00	0x00EA = 234 => 23.4 °C using 0.1 °C/bit scale
Speed	DC 05	0x05DC = 1500 rpm
Frame	ID 0x185, DLC 4	TPDO1 default ID for node 5; verify actual mapping and COB-ID

Mapping entries encode the OD index, sub-index and mapped bit length. Dynamic remapping usually has a required sequence (often temporarily disabling a PDO, clearing mapping count, writing mapping entries, then restoring count/enable state). Follow the actual profile and device manual; do not write mapping records while assuming every device permits dynamic mapping.

14. SDO: parameter access and configuration

In plain English: SDO reads or writes an Object Dictionary entry using a request and a server response.

A Service Data Object (SDO) is used to read or write a parameter in another node's Object Dictionary. The client sends a request, including the index and sub-index; the server returns data, confirms a write, or reports an error (called an abort). SDO is a request/response service, not a single parameter. [R12]

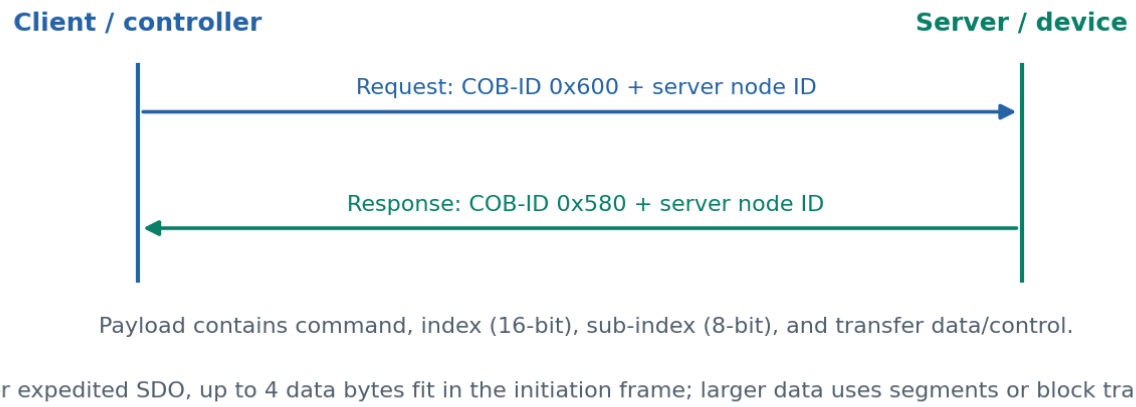


Figure 8. Default SDO request and response identifiers are based on the server node ID.

Typical default expedited upload: read a 32-bit value

Direction	CAN-ID	DLC / bytes	Meaning
Client → server	0x600 + server node ID	8: 40 00 10 00 00 00 00 00	Initiate upload of OD index 0x1000, sub-index 0. Command 0x40 is a typical expedited upload request.
Server → client	0x580 + server node ID	8: 43 00 10 00 78 56 34 12	Illustrative expedited response: command 0x43 indicates 4-byte expedited data; bytes 4..7 = 0x12345678 in little-endian order.

The bytes above are an internally consistent teaching example for a 4-byte expedited upload. Actual response command values depend on data length/transfer type, and object 0x1000 should be read according to its actual

defined type and device documentation. The CAN identifier is separate from index 0x1000.

SDO transfer types

Transfer	Typical use	Payload characteristics
Expedited	Small values, up to 4 data bytes in initiation frame	Single request plus response in the ordinary case
Segmented / normal	Larger values	Initiation then sequenced segments and acknowledgements; each segment carries up to 7 application bytes in Classical CAN SDO
Block	Bulk data where supported	Acknowledges blocks of segments to improve efficiency; implementation support must be verified

SDO aborts and timeouts

On timeout, check that the request COB-ID addresses the correct server, the response COB-ID filter is open, the node is powered/operational enough to serve SDO, and the target index/sub-index exists. If a response contains an abort command, decode the abort code using the applicable CiA/device documentation. Common semantic reasons include unsupported object, invalid sub-index, read-only write, value range error, mapping incompatibility or state-dependent access restriction. Retry only for errors that are transient; repeating an invalid write will not fix it.

SDO versus PDO

	SDO	PDO
Goal	Configuration, diagnostics, parameter access	Fast process/control values
Object addressing	Index + sub-index included in SDO protocol data	OD address not present in payload; defined by mapping
Exchange	Client/server request and response	Producer/consumer broadcast-like data flow
Latency/overhead	Higher, transfer protocol overhead	Lower, one CAN frame per PDO
Confirmation	Service-level response	No per-consumer application response

15. SYNC, TIME and deterministic exchange

In plain English: SYNC helps devices act in step. Good timing still depends on bus load, arbitration, device processing and the chosen cycle.

SYNC lets devices coordinate activities against a common periodic event. The common predefined SYNC COB-ID is 0x080, typically a zero-data Classical CAN frame unless configured otherwise. Synchronous PDO behavior is tied to this event. Depending on the mapping, transmission type and implementation, inputs may be sampled in relation to SYNC and outputs may be applied at a later synchronized point. [R1]

Why it matters

Without synchronization, nodes sample and update at their own independent times, producing phase differences and jitter. SYNC helps align data exchange, but it does not create a hard real-time guarantee by itself: message arbitration, bus load, device processing and clock/cycle configuration still determine end-to-end timing. Configure the cycle period (commonly OD 0x1006 where supported) and synchronous window (0x1007 where supported) according to the system timing budget.

TIME message

CANopen CC includes a TIME protocol for distributing a time-related value. Support and interpretation depend on the applicable profile/version and device. Do not treat it as a general-purpose precision clock synchronization scheme without verifying its definition and accuracy. For many embedded control systems, cycle synchronization and heartbeat are separate concerns: SYNC aligns operations; heartbeat reports node availability/state.

16. EMCY, error register and history

In plain English: EMCY reports a device error event. Decode the payload with the right profile or vendor manual, then follow the system fault policy.

Emergency (EMCY) messages report an internal device error or an error-state transition that the application chooses to announce. In the default CANopen CC connection set the EMCY COB-ID is 0x080 + producer node ID. The data contains an emergency error code, an error-register byte, and manufacturer-specific information within the Classical CAN payload limit. [R1]

Typical payload composition (CANopen CC)

Bytes	Meaning
0–1	Emergency error code (16-bit; standardized or profile/vendor defined according to the error source)
2	Error Register value (commonly reflects OD 0x1001)
3–7	Manufacturer-specific error information (interpret via vendor documentation)

Useful OD objects

Object	Purpose	Caution
0x1001 Error Register	Bit-coded categories of current error condition	Does not replace a detailed vendor fault code
0x1003 Pre-defined Error Field	Error history where implemented	Read the sub-index/count semantics and clear behavior for the device
EMCY message	Event notification over CAN	It is not a CAN error frame and does not automatically stop every machine

A robust application should log timestamp, node ID, EMCY COB-ID, full payload, current NMT state, recent heartbeat/PDO activity and local status. The correct reaction may be warning, controlled stop, fault latch or continued operation; that policy comes from the machine risk analysis and device/application design, not from the presence of an EMCY alone.

17. Heartbeat and node monitoring

In plain English: Heartbeat reports that a node is present and its NMT state. It does not prove that the application data is fresh or correct.

Heartbeat allows a producer to announce its presence and current NMT state periodically. The default CANopen CC identifier is 0x700 + producer node ID. The payload is typically one byte containing the NMT state (boot-up uses zero in its defined context). The producer period is configured at OD 0x1017; consumers configure a timeout appropriate to the producer period and system latency. [R13]

Choosing a timeout

For example, if a heartbeat producer sends every 100 ms, a consumer timeout of 300 ms might tolerate some scheduling jitter and a missed message, but may be too slow for an application requiring a quick stop. This is not a universal rule: choose based on timing analysis, missed-frame behavior, CPU load, bus load, clock tolerance and required fault reaction. Avoid setting timeout so close to the producer period that ordinary jitter produces false trips.

Heartbeat does not prove that the application is healthy.

A node may continue sending heartbeat while its sensor value is stale, a task is deadlocked, the motor drive is disabled, or a control output is not moving. Supervise application-level freshness, watchdog counters, state/command consistency and actual feedback separately. The system should define what happens on a missing heartbeat, unexpected NMT state, repeated boot-up or restored communication.

18. LSS: node-ID and bit-rate configuration

In plain English: LSS can configure node IDs and bit timing on supported devices. A partial network-wide bit-rate change can break communication.

Layer Setting Services (LSS), specified by CiA 305 for CANopen CC, allow an LSS manager to identify/configure LSS-capable devices and change supported parameters such as node ID and bit timing. Default LSS command and response identifiers are 0x7E5 and 0x7E4. LSS capability is not guaranteed simply because a device supports CANopen. [R14]

Practical precautions

- Ensure only the intended server is selected when assigning a node ID. Duplicate node IDs or simultaneous configuration can make normal communication ambiguous.
- Use the identity object fields (vendor ID, product code, revision, serial number) as appropriate to identify a device.
- For bit-rate changes, first configure intended parameters for each device, then command the coordinated switch; any node that fails to switch can isolate itself at the wrong bit rate.
- LSS is not the same as ordinary SDO configuration. SDO access assumes the device can already be reached with a valid node-ID/communication setup.

19. Device profiles: CiA 401 and CiA 402

In plain English: Device profiles make common device types easier to use, but each product supports only the features listed in its EDS and manual.

Communication profile and device profile answer different questions. CiA 301 describes CANopen CC communication services and common objects. Device profiles define application-class behavior so a generic controller can interpret devices more consistently. The exact supported profile revision and optional features must be checked in the EDS and vendor documentation. [R1, R15]

CiA 401: generic I/O

The CiA 401 profile defines standardized application objects for generic digital/analog input-output devices. A controller can use profile-defined objects for values and configuration, while manufacturer-specific objects may expose hardware options, filters, diagnostics or extra channels. Do not assume a given module implements every possible I/O feature.

CiA 402: drives and motion control

CiA 402 standardizes drive state behavior, common controlword/statusword interaction, operating modes and setpoint/feedback objects for supported drive types. The drive state determines which commands are accepted and whether high power may be applied. [R15]

Concept	Meaning
Controlword (commonly 0x6040)	Host command bits used to request drive state transitions or operation behavior
Statusword (commonly 0x6041)	Drive-reported state and flags; decode according to the supported profile
Modes of operation (commonly 0x6060)	Requested operating mode (e.g. profile position/velocity or cyclic modes when supported)
Target / actual values	Objects for velocity, position, torque or other commands and feedback; supported modes vary by drive
RPDO / TPDO mapping	Often used for low-latency command and feedback exchange after configuration

High-level commissioning example

1) Verify drive identity, fault state and vendor-specific preconditions. 2) Read statusword and error information. 3) Clear faults only using the documented procedure. 4) Request the documented shutdown/switch-on/enable operation transitions with the correct controlword sequence. 5) Confirm each transition from statusword before proceeding. 6) Set the supported mode and required limits/setpoints. 7) Confirm feedback is plausible before enabling machine motion. This is a workflow, not a universal hexadecimal sequence; exact transition bits, power-stage interlocks and safety conditions must follow the specific drive manual.

CANopen Safety

Safety-related communication requires the applicable safety protocol/profile, qualified components, hazard analysis, system-level safety architecture, validation and lifecycle process. Ordinary CANopen CC, CAN CRC and heartbeat do not by themselves provide functional safety or message authentication.

20. Complete example network and illustrative trace

In plain English: A real network must start nodes, configure them, enter operation, exchange data, monitor health and react safely to faults.

Example design

Node	Example ID	Role	Example traffic
Controller / NMT manager	Not a CANopen slave ID in this simple illustration	Coordinates startup and application control	NMT, SDO client, heartbeat consumer, PDO consumer/producer as configured
Sensor	5	Measures temperature and speed	Boot-up 0x705; heartbeat 0x705; TPDO1 0x185
I/O module	10	Reads switches, drives outputs	Boot-up/heartbeat 0x70A; RPDO/TPDO according to configuration
Drive	20	Motor drive	Boot-up/heartbeat 0x714; SDO configuration; RPDO command and TPDO status/feedback

Assume all nodes use compatible Classical CAN bit timing, a correctly terminated linear bus, unique node IDs, and an EDS/configuration verified for each product. The PDO mappings below are illustrative.

Operation step by step

- Power-on: MCU clocks and GPIO initialize; transceivers leave standby; CAN controllers enter a configured mode and bit timing/filter/mailbox state is set.
- Each CANopen device completes initialization, sends its boot-up frame using 0x700 + node ID with data 00, then enters pre-operational.
- Manager checks expected nodes and identities, reads required OD entries using SDO, and configures supported PDO COB-IDs/mappings/transmission types.
- Manager sends NMT Start (0x01) to the required node ID or broadcast (node ID 0).
- Runtime: producers send TPDOs when event/timer/SYNC rules trigger them; consumers interpret payload bytes using pre-agreed mapping.
- Manager/consumers monitor heartbeat and application freshness. A missed heartbeat raises a communication fault and triggers an application-defined safe response.
- A local device fault may cause EMCY; application logs error code/register/manufacturer bytes, correlates with local diagnostics and follows the machine fault policy.
- Recovery checks physical communication, node state and device fault status before returning to operational control.

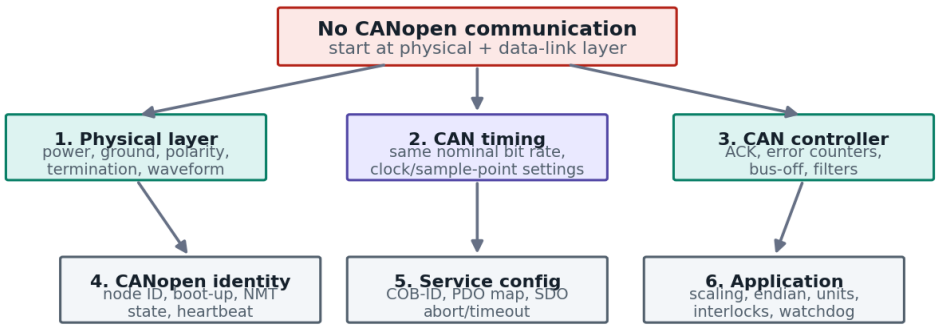
Sample illustrative trace

Time (ms)	CAN-ID	DLC	Data	Interpretation
0	0x705	1	00	Sensor node 5 boot-up
4	0x70A	1	00	I/O node 10 boot-up
8	0x714	1	00	Drive node 20 boot-up
20	0x000	2	01 00	NMT Start broadcast
30	0x605	8	40 00 10 00 …	Illustrative SDO upload request to node 5, read 0x1000:00
32	0x585	8	43 00 10 00 …	Illustrative SDO response from node 5; exact payload is OD-dependent
40	0x185	4	EA 00 DC 05	Sensor TPDO1: 23.4 °C and 1500 rpm under the mapping in Ch. 13
100	0x705	1	05	Heartbeat from node 5 reporting operational state (example state byte)

Trace notes: CAN IDs and data values are demonstration values, not a capture from the Siemens tutorial or a real machine. Heartbeat state byte values should be decoded according to the target CANopen implementation/specification. If the node does not support the assumed object or PDO, the trace must be changed to match the actual device configuration.

21. Debugging and troubleshooting

In plain English: Debug from the wires upward: physical layer, timing/controller, CAN IDs, CANopen state/services, then application data.



Use a CAN trace and an oscilloscope together. A visible frame alone does not prove application data is healthy.

Change one variable at a time; record node ID, nominal bit rate, COB-ID, NMT state, error counters, and timestamps.

Figure 9. Debug in layers: physical → CAN timing/controller → CANopen service/state → application logic.

Symptom	Possible cause	Tests and likely fix
No ACK / transmit errors	No active receiver, bit-rate/timing mismatch, wiring/termination/transceiver issue	Check waveform and bit timing; confirm another powered active node; inspect error counters and controller mode
CAN analyzer sees frames but firmware does not	Acceptance filter, FIFO routing, interrupt/queue failure	Open filters temporarily; inspect peripheral flags; check queue overflow and ISR/task handoff
No boot-up / heartbeat	Wrong node ID/bit rate, device not powered, transceiver standby, CANopen stack not started	Verify power and physical layer, scan expected IDs at known rates, check device LEDs and startup conditions

Symptom	Possible cause	Tests and likely fix
SDO timeout	Wrong request COB-ID, absent server, response filter, state/config issue	Check 0x600 + server node ID request / 0x580 + server node ID response; inspect bus trace
SDO abort response	Unsupported index/sub-index, access type, range, state or mapping error	Decode abort code; compare EDS and manual; read current state/value before writing
PDO absent in pre-op	PDO communication disabled in pre-operational, COB-ID disabled/misconfigured, no trigger	Start node if safe, inspect PDO communication records, transmission type and event/SYNC trigger
PDO present but wrong value	Wrong mapping, byte order, sign extension, scaling, stale data	Verify OD mapping and mapped bit lengths; decode raw bytes independently; compare against known signal
Heartbeat timeout	Producer stopped, bus issue, consumer timeout too short, heartbeat not configured	Check producer 0x1017, actual period, consumer configuration, node state and bus load
Bus-off repeats	Short, bad termination, wrong bit timing, excessive errors, transceiver damage	Capture before fault; log TEC/REC; inspect waveform, wiring and controller recovery sequence
Drive refuses enable	Fault, wrong state transition/controlword/mode, interlock or safety input	Read statusword, error code, mode display and vendor diagnostics; follow drive manual and risk controls
Adding a node breaks network	Duplicate node ID/COB-ID, incompatible bit rate, wrong topology/termination, unexpected CAN FD traffic	Remove new node; compare EDS/DCF, IDs, bit timing and physical-layer topology before reconnecting

Useful tools

- USB-to-CAN adapter and vendor analyzer to capture identifiers, DLC, timestamps, error frames and traffic rate.
- Oscilloscope (preferably differential measurement where appropriate) to inspect dominant/recessive levels, ringing, edge timing and noise.
- Linux SocketCAN utilities such as candump/cansend/can-utils when using a supported adapter and configured interface. Exact commands depend on Linux kernel driver and adapter.
- CANopen configuration tool to inspect EDS/DCF, node state, OD entries, PDO maps, EMCY and heartbeat.
- Firmware logs that correlate CAN events with application states, watchdogs, queue-overflow counts and timestamps.

22. Bus load, latency and system reliability

In plain English: Bus load is the share of time used by CAN frames. Priority, message rate, errors and retransmissions affect delay.

Bus load means how much of the available bus time is used to send frames. It depends on frame length, bit stuffing, message rate, arbitration and retransmissions. A simple calculation is useful for a first estimate, but time-critical systems need a worst-case analysis using the actual message schedule and frame format.

Approximate load example

Assume a standard 11-bit Classical CAN data frame carrying 8 bytes uses approximately 130 bits including typical overhead and inter-frame spacing (illustrative average; actual length varies with bit stuffing and frame fields). At 500 kbit/s, transmission time ≈ 130 / 500,000 = 260 µs. If ten such frames each transmit every 10 ms, the offered utilization is approximately 10 × 260 µs / 10 ms = 26%. This estimate omits retransmissions and exact stuffing; use measured traces and conservative worst-case frame lengths for timing-critical design.

Engineering practices

- Prioritize by deadlines and worst-case response time, not convenience.
- Bound PDO event-trigger bursts and configure event timers/inhibit times where supported.
- Measure bus load under peak operation and fault/retry cases, not only nominal idle behavior.
- Choose heartbeat/consumer timeout and application watchdog thresholds from end-to-end reaction time requirements.
- Use bounded queues and count overrun/overflow events; any loss of frames must have a defined policy.
- Use freshness timestamps or sequence counters in application data when detection of repeated/stale values matters.

23. Security and functional safety

In plain English: CRC is for accidental transmission errors, not security. Functional safety needs a system safety design and applicable standards.

Classical CAN does not natively provide cryptographic source authentication or confidentiality. A node with physical/network access may transmit a valid-looking identifier and payload. CAN CRC detects many accidental transmission errors; it does not prove who sent a message or whether a command is authorized. [R16]

Defense-in-depth ideas

- Segment networks and use gateways with explicit identifier/service allowlists where architecture permits.
- Protect diagnostic and configuration paths; avoid leaving writable SDO parameters exposed without appropriate system-level controls.
- Use secure boot, authenticated/signed firmware update, debug-port lifecycle controls and key management where supported by the overall product.
- Monitor unexpected identifiers, unexpected NMT state transitions, reset/boot-up bursts and implausible application values.
- For safety-related functions, perform hazard analysis and use applicable functional-safety standards, certified safety components and a validated safety lifecycle. Do not infer functional safety from ordinary CANopen or CAN error handling.

Safety caution

A CANopen fault, EMCY message or heartbeat timeout is one input to system behavior, not a complete safety strategy. Machine stop response, braking, torque removal, redundant paths, reset conditions and restart authorization must be designed and validated at system level.

24. Interview question bank with answers

In plain English: A strong interview answer gives the definition, explains why it matters, provides an example and outlines how to debug it.

Use the short answer first, then expand with design trade-offs and a debugging method.

Q1. What is the difference between CAN and CANopen?
CAN provides the data-link/physical communication foundation; CANopen defines higher-layer services, OD objects, NMT, PDO, SDO and profiles.

Q2. Why does a lower CAN ID normally have higher priority?
Dominant 0 overrides recessive 1 during bitwise arbitration; the transmitter that loses arbitration withdraws without corrupting the winning frame.

Q3. What does CAN ACK prove?
At least one active node received the frame correctly at the CAN data-link layer. It does not prove the target application acted on it.

Q4. Why is the bus terminated at both ends?
To match line impedance and reduce reflections. A common high-speed segment uses 120 ohms at each physical end.

Q5. Why can a node transmit but not receive ACK?
No active receiver, incompatible bit timing, wiring fault, transceiver standby/failure, or all other nodes being silent/offline are likely possibilities.

Q6. What is an OD index versus a COB-ID?
OD index/sub-index addresses a device parameter; COB-ID identifies a CAN communication object/frame on the bus.

Q7. SDO versus PDO?
SDO gives confirmed client/server access to an OD entry; PDO sends pre-mapped process bytes with low overhead.

Q8. Why can PDO be absent when heartbeat is present?
The node may be pre-operational, PDO disabled/misconfigured, mapping unsupported, transmission trigger not met, or filters/configuration wrong.

Q9. What does pre-operational mean?
The node is initialized and can generally be configured/diagnosed via SDO, but PDO communication is disabled until operational.

Q10. What is the default heartbeat COB-ID for node 5?
0x705, provided the default connection set is in use.

Q11. What is the default SDO request ID for server node 5?
0x605; the response from that server normally uses 0x585.

Q12. How do you decode a signed 16-bit little-endian PDO value?
Combine low byte + (high byte << 8), cast carefully to int16_t, then apply documented scale/offset and bounds.

Q13. What causes bus-off?
Excessive transmission errors drive the controller transmit error counter to the bus-off threshold. Diagnose root cause before recovery loops.

Q14. Why not do heavy decoding inside an ISR?
It increases interrupt latency and risks starving other real-time work; queue frames and process them in a bounded task.

Q15. What does heartbeat not prove?
It does not prove application task health, sensor freshness, actuator movement or command completion.

Q16. What is PDO mapping?
A configuration that defines which OD values and bit lengths are packed into/out of a PDO payload.

Q17. Why can copying a timing register fail across MCU families?
Clock source, prescaler encoding, segment ranges, sample point, SJW and register conventions differ.

Q18. How would you debug a drive that will not enable?
Read NMT state, SDO access, drive statusword/error, supported mode, controlword sequence, hardware interlocks and safety state; follow vendor manual.

Q19. What is EDS versus DCF?
EDS describes device capabilities/objects; DCF records a configured device's parameter values.

Q20. Does CANopen provide cryptographic security?
Not inherently. Network segmentation, filtering, secure firmware/configuration and system-level security controls are needed.

Q21. What changes with CAN FD?
Payload can reach 64 bytes and data-rate switching is possible; remote frames are not supported. Controller and network compatibility must be verified.

Q22. Can every CANopen node generate SYNC?
No. Producer capability/configuration is specific; many systems designate one SYNC producer.

Q23. What is the best first step when a new node breaks the bus?
Disconnect it safely, verify bit rate/physical layer/termination/node ID/COB-ID and check whether its controller is transmitting incompatible traffic.

Q24. Why might an SDO write fail despite correct CAN-ID?
Object may not exist, be read-only, be out of range, require a different state or have mapping/sequence restrictions.

Q25. How do you distinguish link-layer errors from CANopen errors?
Link-layer issues show controller error counters/error frames/ACK or waveform problems; CANopen errors appear as valid CAN frames with service-level timeouts, aborts, unexpected state or mapping issues.

Scenario questions: answer with a repeatable workflow

- **No node in trace:** confirm power/transceiver/bit rate, termination and analyzer configuration; check boot-up ID range.
- **Frames visible, data wrong:** verify COB-ID, DLC, mapping, endianness, sign, scale, units and producer freshness.
- **SDO timeout:** correlate request and response IDs, verify server node ID and filters, check NMT state, then inspect service aborts.
- **Heartbeat stopped:** check the producer's own bus/error state, power, watchdog/reset, consumer timeout and bus load.
- **Repeated bus-off:** capture errors and waveform, isolate likely physical causes, then validate controller timing and recovery policy.
- **Motor drive refuses state change:** read statusword and error, confirm supported mode and required transitions/interlocks; never guess bit masks on a real machine.

25. Practical exercises with worked solutions

In plain English: Try each calculation and trace first, then compare your working with the answer and state your assumptions.

Exercise 1 — Identifier arithmetic

A CANopen CC node has node ID 7 and uses the default TPDO1. What is the COB-ID?

Solution: 0x180 + 0x07 = `0x187`. Verify configured COB-ID; defaults are not guaranteed after commissioning.

Exercise 2 — Decode PDO payload

Given ID 0x185, DLC 4, bytes EA 00 DC 05, use the Chapter 13 mapping: signed 16-bit temperature at 0.1 °C/bit, then unsigned 16-bit speed in rpm.

Solution: temperature raw = 0x00EA = 234 -> 23.4 °C. Speed = 0x05DC = 1500 rpm. Confirm mapping, sign, scale and byte order on the actual device.

Exercise 3 — Why does transmitter report ACK error?

A controller sends frames but reports no ACK. List likely causes and tests.

Solution: confirm another node is powered and active; check nominal bit rate/sample timing, CAN_H/CAN_L, transceiver standby, termination and waveform. An analyzer in listen-only mode may not ACK; some tools require active mode. Do not conclude “the receiver is present” simply because its power LED is on.

Exercise 4 — Distinguish ID from OD address

You send an SDO request on CAN-ID 0x605 to access index 0x1000, sub-index 0. Are 0x605 and 0x1000 interchangeable?

Solution: No. 0x605 routes the SDO request to the server node under the default connection set; 0x1000:00 is the address of the parameter inside that server’s OD.

Exercise 5 — Startup state diagnosis

A node sends boot-up and heartbeat but no PDO. What do you check?

Solution: verify heartbeat state is operational; pre-operational commonly disables PDO. Then inspect PDO COB-ID enable bits, mapping, transmission type/event, NMT command, device support, and consumer hardware/software filters.

Exercise 6 — Approximate bus load

At 500 kbit/s, estimate load for 20 frames each consuming 130 bits on the wire every 20 ms.

Solution: each frame time ≈ 130/500,000 = 260 µs. Twenty frames per 20 ms consume 5.2 ms per 20 ms -> 26% approximate utilization. Exact load depends on actual bit count, stuffing, retransmission and other traffic.

Exercise 7 — Build a troubleshooting checklist

Write an order of operations for a sensor that stopped updating.

Suggested answer: verify power/transceiver/termination; inspect CAN waveform and controller errors; confirm bit rate and ID visibility; check boot-up/heartbeat/NMT state; inspect TPDO COB-ID/mapping/trigger; decode raw data and scaling; correlate with sensor diagnostics and application freshness counter.

26. Field checklists and revision sheets

In plain English: Use these checklists during commissioning and troubleshooting, but confirm defaults and device behavior in real traces.

Network bring-up checklist

- [ ] Read the target controller, transceiver, device manual, EDS/DCF and applicable CiA profile version.
- [ ] Confirm one consistent nominal bit rate and valid bit-timing margins for all nodes.
- [ ] Verify CAN_H/CAN_L polarity, ground/reference and any shield policy.
- [ ] Confirm exactly the intended two end terminations on a conventional high-speed trunk.
- [ ] Assign unique node IDs; check COB-ID collisions and configured IDs.
- [ ] Confirm boot-up messages and identity object data for all expected nodes.
- [ ] Configure and verify PDO COB-IDs, mapping records, transmission types and scaling.
- [ ] Verify NMT startup and operational transitions; check PDO behavior in operational state.
- [ ] Configure heartbeat producer/consumer and application watchdogs.
- [ ] Test SDO timeout/abort behavior and EMCY/error logging.
- [ ] Test unplugged node, bus-off, stale data and recovery behavior in a controlled safe setup.
- [ ] Record nominal bus load, highest observed load, queue overflow counters and timing margins.

Quick comparison sheets

Question	Quick answer
CAN identifier vs node ID	CAN ID identifies the frame/priority; a CANopen node ID is a higher-layer device identifier often embedded in default COB-IDs.
CAN ACK vs SDO response	CAN ACK is data-link reception; SDO response is service-level result.
SDO vs PDO	SDO = parameter access and confirmed client/server exchange; PDO = mapped real-time data with low overhead.
Heartbeat vs EMCY	Heartbeat reports availability/NMT state cyclically; EMCY reports a device error event.
SYNC vs heartbeat	SYNC aligns synchronous communication; heartbeat supervises node availability/state.
CANopen CC vs FD	CC uses Classical CAN (up to 8-byte payload); FD uses CAN FD (up to 64-byte payload) and a distinct protocol profile.
EDS vs DCF	EDS describes a device; DCF records a configured device’s values.

One-page interview revision

Physical: linear trunk, two end terminators, transceiver enable, common bit timing. **CAN data link:** frame identifier, arbitration, CRC, ACK, error counters and bus-off. **CANopen:** OD index/sub-index, COB-ID, NMT state, SDO read/write, PDO mapping, SYNC, EMCY, heartbeat. **Debug order:** voltage/waveform → controller status/timing → IDs/filters → boot-up/NMT → service config → mapping/scaling → application health. **Never assume:** default IDs are configured; a CAN ACK proves app success; heartbeat proves values are fresh; CRC provides authentication; a generic CiA 402 command sequence fits every drive.

Recommended study roadmap

Stage	Study and practice
1. Fundamentals	Frame fields, dominant/recessive, arbitration, ACK, termination; decode traces in a CAN analyzer.
2. Firmware	Initialize a chosen MCU CAN peripheral using its reference manual; implement bounded TX/RX and error logging.
3. CANopen basics	Inspect EDS/DCF, node ID, NMT states, SDO uploads/downloads and heartbeat.
4. Process data	Create a PDO mapping, decode signed/scaled signals, test SYNC and bus load.
5. Diagnostics	Inject controlled faults; observe EMCY, SDO abort, heartbeat timeout, bus-off and recovery.
6. Advanced	Study worst-case latency, drive profile, safety architecture, cybersecurity and the exact applicable CiA specifications.

27. Glossary and reference material

In plain English: Use the glossary to expand short forms and the references to check details for the exact controller and device.

Term	Meaning
ACK	Acknowledgement slot in a CAN frame
CAN	Controller Area Network
CAN CC	Classical CAN data-link communication
CAN FD	CAN with Flexible Data-rate
CANopen CC / FD	CANopen application/communication profile variants for Classical CAN / CAN FD
COB / COB-ID	Communication Object / Communication Object Identifier
CRC	Cyclic Redundancy Check
DCF	Device Configuration File
DLC	Data Length Code
EDS	Electronic Data Sheet
EMCY	Emergency message service in CANopen
HAL	Hardware Abstraction Layer
LSS	Layer Setting Services
NMT	Network Management
OD	Object Dictionary
PDO / RPDO / TPDO	Process Data Object / Receive PDO / Transmit PDO
REC / TEC	Receive Error Counter / Transmit Error Counter
RTR	Remote Transmission Request bit in Classical CAN frame formats
SDO	Service Data Object
SYNC	Synchronization object
TQ	Time Quantum
TPDO/RPDO mapping	Definition of values and bit lengths carried in a PDO payload

References

The Siemens PDF is the introductory baseline. Use the current CiA guidance and the exact product manuals for implementation details.

[R1] Siemens AG, CANopen Tutorial, Version 2.0, July 2019, Entry ID 109479771.
https://support.industry.siemens.com/cs/ww/en/view/109479771

[R2] CAN in Automation (CiA), CANopen overview and communication profile.
https://www.can-cia.org/can-knowledge/canopen

[R3] CAN in Automation (CiA), CAN CC (Classical CAN) knowledge base.
https://www.can-cia.org/can-knowledge/can-cc

[R4] Texas Instruments, CAN physical layer / transceiver design information.
https://www.ti.com/interface/can-transceivers/overview.html

[R5] ISO 11898-2, High-speed medium access unit physical layer (consult licensed standard and transceiver datasheet).
https://www.iso.org/standard/67244.html

[R6] Bosch Semiconductors, CAN FD protocol overview.
https://www.bosch-semiconductors.com/products/ip-modules/can-protocols/can-fd/

[R7] CiA 301, CANopen application layer and communication profile (CiA knowledge page / specification catalogue).
https://www.can-cia.org/can-knowledge/canopen

[R8] CiA, CANopen FD profile and CiA 1301 information.
https://can-cia.org/cia-groups/technical-documents

[R9] STMicroelectronics, AN5348, Introduction to FDCAN peripherals for STM32 MCUs, Rev. 6, January 2026.
https://www.st.com/content/st_com/en/technical-documents/an5348-introduction-to-fdcan-peripherals-for-stm32-mcus.html

[R10] CiA, PDO cross communication/default CAN-ID connection-set note.
https://www.can-cia.org/fileadmin/cia/documents/publications/cnlm/december_2024/24-4_cnlm.pdf

[R11] CiA, PDO protocol.
https://www.can-cia.org/can-knowledge/pdo-protocol/

[R12] CiA, SDO protocol.
https://www.can-cia.org/can-knowledge/sdo-protocol

[R13] CiA, Error control protocols (heartbeat, boot-up and legacy guarding).
https://www.can-cia.org/can-knowledge/error-control-protocols

[R14] CiA 305, Layer Setting Services (LSS).
https://www.can-cia.org/can-knowledge/cia-305-layer-setting-services-lss

[R15] CiA 402 series, CANopen drive and motion-control profile.
https://www.can-cia.org/can-knowledge/cia-402-series-canopen-device-profile-for-drives-and-motion-control

[R16] Siemens Industry Online Support, industrial security information and secure deployment guidance.
https://www.siemens.com/industrialsecurity

[R17] CiA, Network Management (CANopen CC NMT states and PDO state rules).
https://www.can-cia.org/can-knowledge/network-management/

[R18] Bosch Semiconductors, CAN Protocols including CAN XL and CAN FD.
https://www.bosch-semiconductors.com/products/ip-modules/can-protocols/

Verification and limitations

Verification performed for this edition: checked CANopen CC basics against the Siemens tutorial and current CiA guidance; checked CAN frame/arbitration/ACK concepts against CiA CAN CC material; updated PDO Object Dictionary index ranges to the current CiA ranges; checked default COB-ID arithmetic, SDO direction and payload bytes, NMT state behavior, heartbeat/LSS identifiers, bit-timing arithmetic, PDO decoding and bus-load arithmetic. The generated PDF is also rendered and reviewed for layout. This is an educational handbook, not a conformance test suite. Exact controller registers, device options, standards revisions and safety reactions must still be checked against the target product and applicable standards.

End of handbook